

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут
Кафедра інформаційної безпеки

ЗВІТ

з дисципліни «Математичне моделювання систем і процесів»

Лабораторна робота № 3 — Моделювання систем з використанням мереж Петрі

Варіант 3

Виконав: Вовкотруб Олександр Віталійович

Група: ФБ-61мн

Перевірив: доц. Смирнов С.

Зміст

1	Мета роботи	2
2	Постановка задачі	2
3	Хід виконання	2
3.1	1. Мережа Петрі варіанта 3 — зчитування й перевірка транскрипції	2
3.2	2. Граф досяжних розміток	3
3.3	3. Р-інваріанти	4
3.4	4. Синтез — задача 3: обідаючи філософи із запрошеннями	5
4	Результати	10
5	Висновки	10
6	Джерела	11

1 Мета роботи

Ознайомлення з формалізмом мереж Петрі (МП) як засобом моделювання взаємодіючих паралельних і конкурентних процесів: побудова графа досяжних розміток заданої мережі та структурний доказ її властивостей через Р-інваріанти (аналіз), і синтез власної моделі МП для задачі про обідаючих філософів разом із перевіркою того, який саме ефект на поведінку моделі має конкретна структурна зміна мережі (синтез).

2 Постановка задачі

Робота складається з двох завдань за варіантом 3.

Завдання з аналізу. Задано мережу Петрі № 3 рисунком («Варіанти мереж Петрі», с. 6 умови). Побудувати граф досяжних розміток, визначити межі позицій і наявність тупиків.

Завдання з синтезу. Для варіанта 3 з переліку задач синтезу умови — задача 3: «Перетворення задачі 2 про обідаючих філософів для випадку, коли за стіл запрошують одного з філософів, який не перешкоджає загальному поглинанню їжі обідаючими.» Задача 2, від якої відштовхуємось: «Перетворення задачі 1 про обідаючих філософів для випадку, коли ліву та праву вилки беруть по черзі (не одночасно).» Задача 1 (вихідна): п'ять філософів по колу, кожен для трапези бере одночасно дві виделки, що лежать між тарілками. Побудувати відповідну модель МП, дослідити її граф досяжності й тупики.

Повний текст умови — `task/README.md`, оригінал — `task/ЛР№3.pdf`.

3 Хід виконання

3.1 1. Мережа Петрі варіанта 3 — зчитування й перевірка транскрипції

Мережа задана рисунком. Нижче — вирізка оригіналу (ліворуч) і та сама мережа, перемальована засобами Graphviz з явними іменами $p1$ – $p4$, $t1$ – $t3$ та розміткою μ_0 (праворуч):

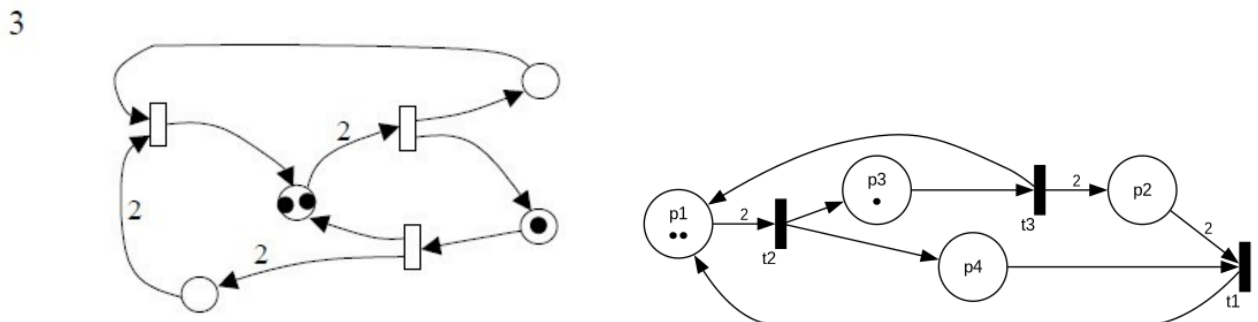


Рисунок 1: Ліворуч — вирізка оригінального рисунка мережі варіанта 3 (с. 6 умови); праворуч — та сама мережа, перемальована з явними іменами позицій і переходів; фішки μ_0 показані крапками.

Дуги звірено вручну: для кожного з трьох прямокутників (переходів) на оригіналі простежено кожну вхідну й вихідну стрілку разом з її вагою (мітка «2», де вона проставлена; інакше вага дорівнює 1 за замовчуванням) і зіставлено з перемальованою мережею. Лівий прямокутник має вхід вагою 2 від нижнього лівого кружка й вхід вагою 1 від верхнього правого, вихід — у центральний кружок з двома фішками; верхній правий прямокутник бере з центрального кружка вагою 2 й віддає по одному в верхній правий і нижній правий кружки; нижній прямокутник бере одну фішку з нижнього правого кружка (де вона й намальована, ●) і віддає одну назад у центр та дві — у нижній лівий кружок. Це узгоджується з транскрибованою таблицею переходів рядок у рядок, а рівно три мітки «2» на оригіналі відповідають рівно трьом дугам вагою 2 в таблиці — по одній на кожен перехід. Докладніше про методику звірки — у `notes.md`.

Перехід	Вхід	Вихід
t1	p4(1), p2(2)	p1(1)
t2	p1(2)	p4(1), p3(1)
t3	p3(1)	p1(1), p2(2)

```
PLACES_V3 = ["p1", "p2", "p3", "p4"]
TRANS_V3 = {
    "t1": ({ "p4": 1, "p2": 2 }, { "p1": 1 }),
    "t2": ({ "p1": 2 }, { "p4": 1, "p3": 1 }),
    "t3": ({ "p3": 1 }, { "p1": 1, "p2": 2 }),
}
M0_V3 = (2, 0, 1, 0)
```

Початкова розмітка $\mu_0 = (2, 0, 1, 0)$ у порядку (p_1, p_2, p_3, p_4) : p1 — центральний кружок (2 фішки, збігається з ●● на оригіналі), p3 — кружок праворуч знизу (1 фішка, ●), p2 і p4 — порожні кружки.

3.2 2. Граф досяжних розміток

Граф побудований обходом у ширину (`PetriNet.reachability_graph`) від μ_0 : на кожному кроці для поточної розмітки перевіряються всі дозволені переходи (`enabled`), кожен виконується (`fire`), і нова розмітка додається до черги, якщо ще не зустрічалась. Обхід завершується природно, без штучного обмеження глибини чи числа кроків, — граф побудований повністю, а не є вибіркою.

№	$\mu = (p_1, p_2, p_3, p_4)$	Дозволені переходи
1 (μ_0)	(2, 0, 1, 0)	t2, t3
2	(0, 0, 2, 1)	t3

№	$\mu = (p_1, p_2, p_3, p_4)$	Дозволені переходи
3	(3, 2, 0, 0)	t2
4	(1, 2, 1, 1)	t1, t3
5	(2, 4, 0, 1)	t1, t2
6	(0, 4, 1, 2)	t1, t3
7	(1, 6, 0, 2)	t1

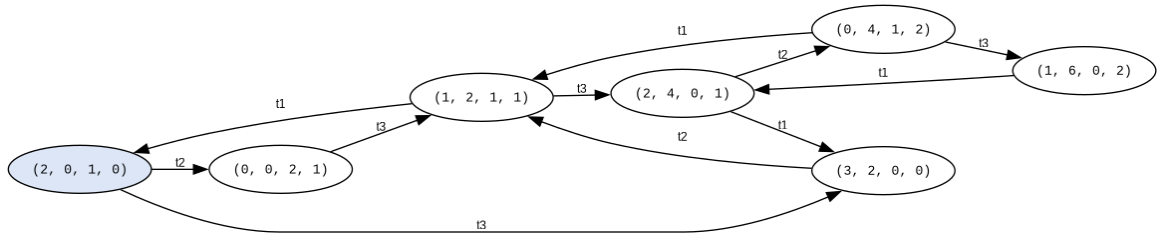


Рисунок 2: Граф досяжних розміток мережі варіанта 3: 7 розміток, 11 дуг; початкова розмітка виділена кольором

Усього 7 досяжних розміток і 11 дуг між ними. У кожному з 7 рядків таблиці хоч один перехід дозволений — жодна розмітка не тупикова (тупикових розміток = 0). Оскільки граф побудований вичерпно (обхід у ширину без обриву, простір станів скінченний), відсутність тупиків тут — не спостереження на вибірці кроків, а повний доказ: серед усіх досяжних з μ_0 станів мережі варіанта 3 тупика немає.

Межі позицій — максимум фішок у кожній позиції серед усіх 7 розміток: $p_1 \leq 3$ (рядок 3), $p_2 \leq 6$ (рядок 7), $p_3 \leq 2$ (рядок 2), $p_4 \leq 2$ (рядки 6 і 7). Усі межі скінченні — мережа **обмежена**, і найбільша з них ($k = 6$, позиція p_2) визначає ступінь обмеженості: мережа **6-обмежена**. Оскільки $k = 6 > 1$, мережа **не є безпечною** (безпечна мережа вимагає $k \leq 1$ у кожній позиції).

3.3 3. Р-інваріанти

Матриця інцидентності $C = O - I$ (стовпці — переходи t1, t2, t3, рядки — позиції $p_1 \dots p_4$), обчислена автоматично (net.incidence()):

$$C = \begin{bmatrix} 1 & -2 & 1 \\ -2 & 0 & 2 \\ 0 & 1 & -1 \\ -1 & 1 & 0 \end{bmatrix}$$

Р-інваріант — цілочисловий вектор x , для якого $x^T C = 0$; тоді зважена сума фішок $x^T \mu$ **однакова в будь-якій досяжній розмітці**, незалежно від того, який перехід і скільки разів спрацював. net.p_invariants() знаходить базис нуль-простору C^T

(`sympy.Matrix.nullspace()`) і зводить його до цілих чисел. Для варіанта 3 знайдено два базисні інваріанти, перевірені не лише на μ_0 , а на всіх 7 досяжних розмітках:

```
for inv in invariants:
    expr = " + ".join(f"{w}·{p}" for w, p in zip(inv, PLACES_V3) if w)
    values = {sum(w * m[i] for i, w in enumerate(inv)) for m in nodes}
    assert len(values) == 1
```

Інваріант (базис, повернутий <code>nullspace()</code>)	Стала
$2p_1 + 1p_2 + 4p_3$	8
$2p_1 - 1p_2 + 4p_4$	4

Чому саме перший рядок доводить обмеженість p_2 . З першого рівняння $2p_1 + p_2 + 4p_3 = 8$ і невід’ємності $p_1, p_3 \geq 0$ одразу випливає $p_2 \leq 8$ — структурна межа, яка тримається на самій алгебрі мережі й чинна для будь-якої досяжної розмітки, навіть тієї, що перебором іще не знайдено. Це доведення **не залежить від перебору графа**: воно впливає лише з того, що зважена сума фішок стала, а всі ваги в ній невід’ємні, причому вага при p_2 додатна.

Фактична межа $p_2 \leq 6$, знайдена в розділі 2 перебором графа досяжності, тісніша за $p_2 \leq 8$ з інваріанта — і це не помилка, а межа самого методу. Інваріант гарантує, що *кожна* досяжна розмітка задовольняє його рівнянню, але не гарантує зворотного: не кожна невід’ємна ціла розмітка, яка задовольняє рівнянням, насправді досяжна з μ_0 . Контрприклад: $\mu = (0, 8, 0, 3)$ задовольняє обидва рівняння з тими самими сталими, що й μ_0 ($2 \cdot 0 + 8 + 4 \cdot 0 = 8$, $2 \cdot 0 - 8 + 4 \cdot 3 = 4$), але серед 7 досяжних розміток його немає. Отже інваріанти доводять обмеженість **структурно**, без побудови графа, а точну межу $p_2 \leq 6$ дає лише вичерпний перебір графа досяжності — це два різні, взаємодоповняльні докази, і плутати їх не можна.

Якщо додати обидва рівняння інваріантів і поділити на 4, вийде читабельніша, але похідна (не третя незалежна) форма: $p_1 + p_3 + p_4 = 3$ — звідси одразу видно, що жодна з p_1, p_3, p_4 не перевищує 3 (кожна входить з додатною вагою в сталу суму). У звіті як обчислений базис наведено саме пару $[2, 1, 4, 0]$ і $[2, -1, 0, 4]$, а не цю похідну форму.

3.4 4. Синтез — задача 3: обідаючи філософи із запрошеннями

Мій варіант синтезу — задача 3 з переліку умови (формулювання наведене в постановці задачі). Щоб було видно саме *перетворення* умови, а не лише кінцевий результат, побудовано три моделі поспіль — задача 1 (вихідна), задача 2 (те, що перетворюємо) і задача 3 (мій варіант), кожна для $n = 3$ і $n = 5$ філософів. Філософи сидять по колу: філософ i використовує виделки i та $(i + 1) \bmod n$.

Задача 1 — обидві виделки одночасно. Філософ i бере обидві виделки одним переходом `take_i` (атомарно) і повертає їх одним `put_i`:

```
transitions[f"take{i}"] = (
    {f"think{i}": 1, f"fork{i}": 1, f"fork{right}": 1}, {f"eat{i}": 1})
```

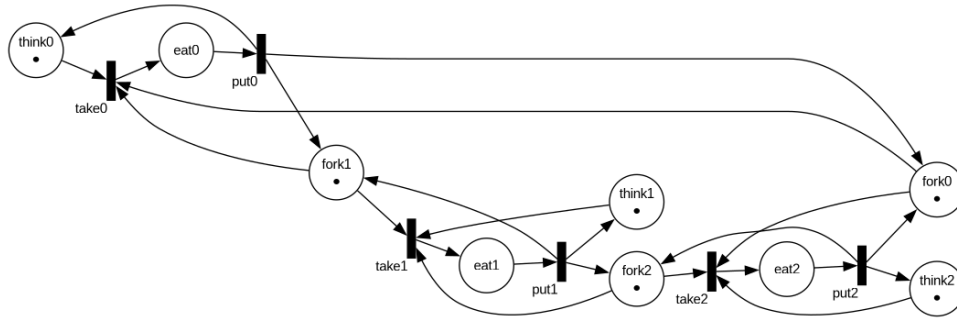


Рисунок 3: Модель задачі 1 ($n = 3$): обидві виделки беруться одночасно

Атомарність узяття не залишає проміжного стану, у якому філософ тримає лише одну виделку, — застрягнути ні на чому.

Задача 2 — виделки по черзі. Ліва й права виделки беруться окремими переходами takeL_i / takeR_i через проміжну позицію left_i:

```
transitions[f"takeL{i}"] = ({f"think{i}": 1, f"fork{i}": 1}, {f"left{i}": 1})
transitions[f"takeR{i}"] = ({f"left{i}": 1, f"fork{right}": 1}, {f"eat{i}": 1})
```

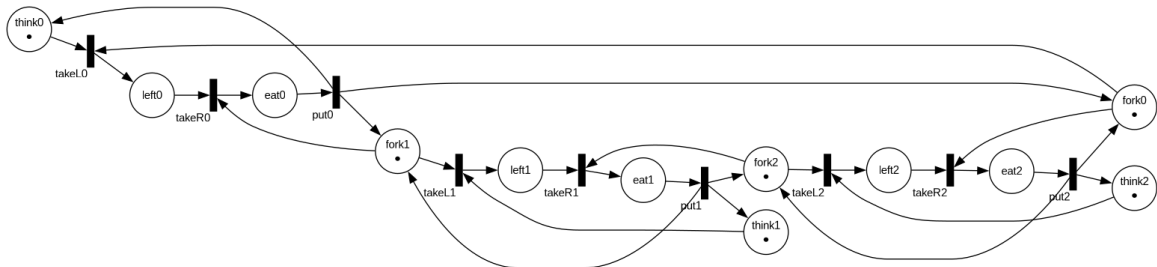


Рисунок 4: Модель задачі 2 ($n = 3$): виделки беруться по черзі, з'являється позиція left_i

Саме проміжний стан left_i і робить можливим класичний тупик обідаючих філософів (нижче).

Задача 3 (мій варіант) — доступ за запрошенням. Додано позицію invite — семафор: узяти ліву виделку (takeL_i) можна лише витративши запрошення, а повертається воно по завершенні трапези (put_i):

```
if name.startswith("takeL"):
    pre["invite"] = 1
if name.startswith("put"):
    post["invite"] = 1
```

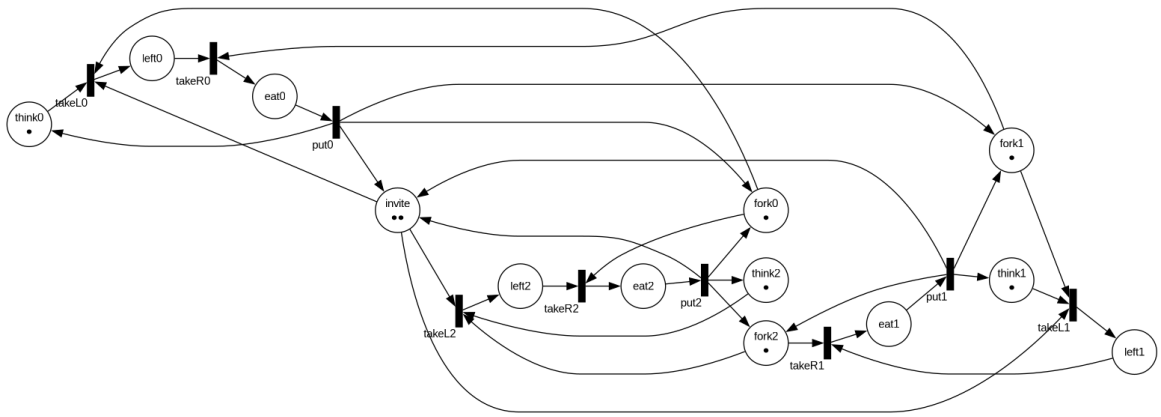


Рисунок 5: Модель задачі 3 ($n = 3$): додана позиція invite з початковою розміткою $n-1$

Скільки запрошень: $n - 1$, а не 1. Умову можна прочитати двояко:

- буквально «запрошують **одного**» → в invite кладеться 1 фішка. Але тоді за столом одночасно може їсти щонайбільше один філософ — а це прямо суперечить продовженню тієї самої фрази: «**не перешкоджає загальному поглинанню їжі обідаючими**». Один їдець одночасно — це черга, а не загальне поглинання їжі;
- отже фішок має бути стільки, щоб паралельне харчування зберігалось, а тупик усе одно зникав: $n - 1$. Це стандартний семафорний розв'язок задачі про обідаючих філософів — якщо за столом одночасно щонайбільше $n - 1$ філософів, завжди лишається хоча б один філософ, чий сусід ще не сидить за столом (не взяв запрошення), тож ланцюжок очікування «кожен тримає ліву й чекає на праву» не може замкнутися в коло одразу на всіх n філософах.

Третій, незалежний аргумент дає **задача 4** з того самого переліку синтезу: «Перетворення задачі 3 для випадку, коли запрошення до столу **необов'язкові для розгляду філософом**». Це формулювання явно передбачає, що в задачі 3 запрошення — обов'язковий, постійно діючий механізм доступу до столу (задача 4 лише знімає цю обов'язковість), а не одноразова подія «когось один раз запросили». Отже invite діє як семафор, що витрачається на вхід ($takeL_i$) і повертається на вихід (put_i), з початковою кількістю $n - 1$, а не як фішка, видана один раз і забута.

Тупик задачі 2 — показано, а не задекларовано. Тупик знайдено не оглядом, а програмно: серед 14 досяжних розміток (для $n = 3$) знайдено ту єдину, де жоден перехід не дозволений, і відновлено найкоротший шлях до неї обходом у ширину по вже побудованих дугах графа. Послідовність спрацювань:

$$takeL0 \rightarrow takeL1 \rightarrow takeL2$$

Тупикова розмітка: $left0 = left1 = left2 = 1$, усі інші позиції — 0 (зокрема всі три $fork_i$ порожні, усі eat_i і $think_i$ порожні). Кожен філософ по черзі взяв свою ліву виделку; після третього кроку всі три виделки розібрані як «ліві», і жодному філософу вже нема чим

узяти праву — переходи takeR_i заблоковані назавжди (put_i теж, бо ніхто не їсть). Це і є класичний тупик обідаючих філософів, відтворений мережею Петрі буквально.

Що саме прибрали запрошення. Графи досяжності для $n = 5$ завеликі, щоб намалювати їх читабельно (82 і 81 вершина для задач 2 і 3 відповідно) — тому нижче показано модель, зменшену до $n = 3$ філософів; вона має ту саму структуру побудови (ті самі позиції й переходи, той самий семафор), лише менший граф. Числа для $n = 5$ наведено в таблиці нижче — вони отримані тим самим кодом, лише не намальовані.

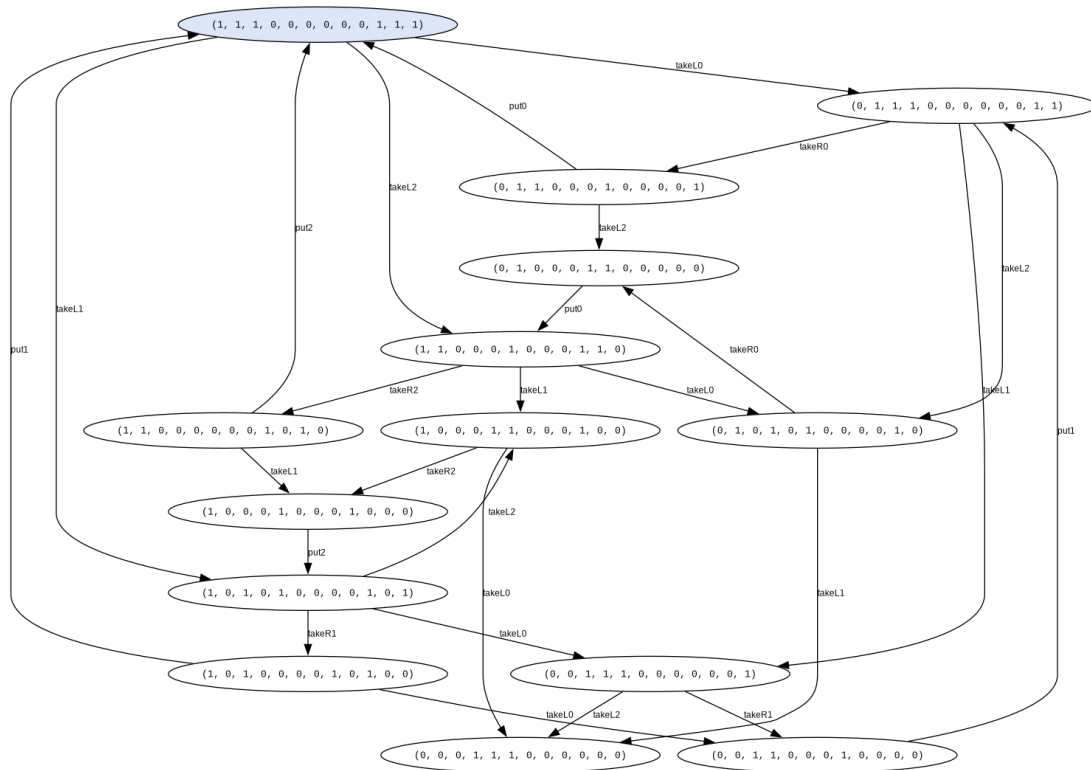


Рисунок 6: Граф досяжності задачі 2 ($n = 3$, по черзі): 14 розміток, 27 дуг, 1 тупик

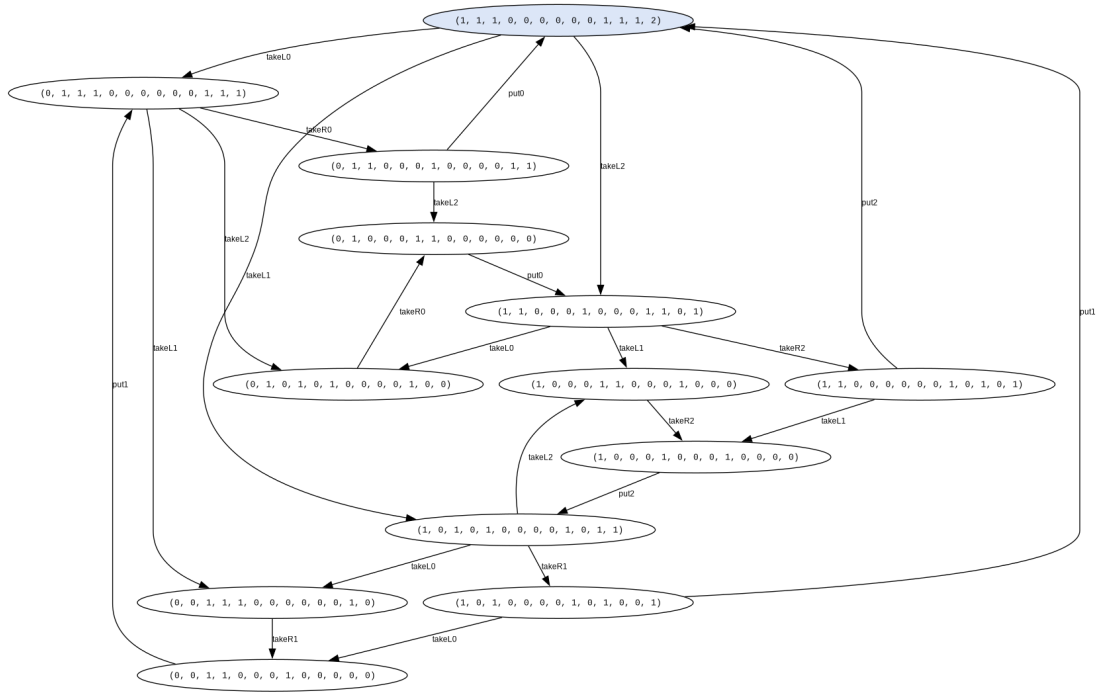


Рисунок 7: Граф досяжності задачі 3 ($n = 3$, із запрошеннями): 13 розміток, 24 дуги, тупиків немає

Числовий підсумок порівняння — у таблиці нижче: граф задачі 3 має рівно на одну вершину й рівно на три дуги менше за граф задачі 2, і зникла вершина — саме тупикова розмітка з попереднього пункту ($\text{left0}=\text{left1}=\text{left2}=1$), у яку сходяться всі три переходи takeL_i .

Модель	$n = 3$: розміток / дуг / тупиків	$n = 5$: розміток / дуг / тупиків
Задача 1 (обидві виделки одночасно)	4 / 6 / 0	11 / 30 / 0
Задача 2 (по черзі)	14 / 27 / 1	82 / 265 / 1
Задача 3 (із запрошеннями)	13 / 24 / 0	81 / 260 / 0

Головний результат: $14 \rightarrow 13$ і $82 \rightarrow 81$ — семафор запрошень прибирає з графа досяжності рівно **одну** розмітку за кожного n , і це саме та розмітка, у якій усі філософи тримають ліву виделку й чекають на праву (описана вище). Жодна інша поведінка не втрачена — усе, що було досяжним у задачі 2, крім цього єдиного тупика, лишається досяжним і в задачі 3.

Контрольна перевірка ($\text{build_task3}(n, \text{invites}=n)$, тобто без обмеження запрошень): якщо дозволити всім n філософам запрошення одночасно, тупик задачі 2 повертається (тупиків знову 1) — це підтверджує, що причина усунення тупика саме межа $n - 1$, а не сам факт наявності позиції invite .

4 Результати

Аналіз мережі Петрі варіанта 3.

Показник	Значення
Досяжних розміток	7
Дуг у графі досяжності	11
Тупикових розміток	0
Межі позицій	$p_1 \leq 3, p_2 \leq 6, p_3 \leq 2, p_4 \leq 2$
Ступінь обмеженості k	6 (не безпечна, $k > 1$)
P-інваріант 1	$2p_1 + p_2 + 4p_3 = 8$
P-інваріант 2	$2p_1 - p_2 + 4p_4 = 4$

Синтез — задача 3 (обідаючи філософи).

Показник	Значення
Послідовність до тупика (задача 2, $n = 3$)	takeL0 -> takeL1 -> takeL2
Довжина шляху до тупика	3
Розміток: задача 2 → задача 3 ($n = 3$)	$14 \rightarrow 13$
Тупиків: задача 2 → задача 3 ($n = 3$)	$1 \rightarrow 0$
Розміток: задача 2 → задача 3 ($n = 5$)	$82 \rightarrow 81$
Тупиків: задача 2 → задача 3 ($n = 5$)	$1 \rightarrow 0$
Задача 3 з invites=n (контроль)	тупиків 1

5 Висновки

Аналіз (мережа варіанта 3). Граф досяжності — 7 розміток, 11 дуг — побудований вичерпно, тому висновок про відсутність тупиків остаточний, а не спостереження на вибірці: жодна з 7 розміток не тупикова. Межі позицій ($p_1 \leq 3, p_2 \leq 6, p_3 \leq 2, p_4 \leq 2$) роблять мережу 6-обмеженою і, оскільки $k = 6 > 1$, небезпечною. Два P-інваріанти, знайдені через нуль-простір матриці інцидентності, доводять обмеженість позицій структурно, незалежно від побудови графа: з $2p_1 + p_2 + 4p_3 = 8$ одразу випливає $p_2 \leq 8$. Це доведення **сильніше** за перебір у тому сенсі, що спирається лише на алгебру мережі (матрицю інцидентності) і лишається чинним, навіть якщо граф досяжності був завеликим для побудови; водночас воно **слабше** в точності — контрприклад $(0, 8, 0, 3)$ задовольняє обидва рівняння, але недосяжний, — тож $p_2 \leq 6$ дає лише перебір. Ці два докази доповнюють, а не замінюють один одного: інваріанти — для структурної гарантії за будь-якого розміру графа, перебір — для точної межі.

Синтез (задача 3). Три моделі, побудовані послідовно (атомарне взяття виделок → взяття по черзі → взяття по черзі із семафором запрошень), показують саме механізм виникнення й усунення тупика: проміжна позиція left_i (задача 2) вперше уможливорює

тупик «усі тримають ліву, чекають на праву», а позиція-семафор `invite` з початковою розміткою $n - 1$ (задача 3) прибирає з графа досяжності **рівно цю одну** розмітку — $14 \rightarrow 13$ при $n = 3$ і $82 \rightarrow 81$ при $n = 5$ — і жодну іншу, зберігаючи паралельне харчування кількох філософів одночасно. Контрольний запуск з `invites=n` повертає тупик, підтверджуючи, що причина усунення — саме межа $n - 1$, а не сама наявність позиції-семафора.

Обидва завдання спираються на одну властивість мереж Петрі — **недетермінізм** правила спрацювання: якщо дозволено кілька переходів, а фішок вистачає лише на один, вибір випадковий. Це саме та властивість, яку природно вимагає моделювання конкурентних процесів, чий порядок дій наперед не заданий (кілька філософів, що діють незалежно): мережа Петрі не нав'язує жодного конкретного розкладу дій, а її граф досяжності охоплює **всі** можливі чергування одразу. Тому тупик задачі 2, що виникає лише за певного (хай і ймовірного) чергування дій філософів, не губиться в перевірці одного «типового» сценарію — він неминуче виявляється в повному графі досяжності, який і був побудований у розділі 4.

6 Джерела

1. Умова лабораторної роботи № 3 — `task/ЛР№3.pdf`, викладена також у `task/README.md`.
2. Документація `sympy` (нуль-простір матриці для P-інваріантів) — <https://docs.sympy.org/latest/index.html>.
3. Документація `graphviz` (Python-бібліотека рендерингу графів) — <https://graphviz.readthedocs.io/>.